Advanced Computational Physics, University of Strathclyde, WS 2026,
Dr. Ben Hourahine

Assignment 3

Jörn Wilhelm
Glasgow, 18. March 2026

Running the code yields the three plots of the greens function for points $(0.5, 0.5)$, $(0.02, 0.5)$ and $(0.02, 0.02)$. These are shown in Figure 1. Note, that the charge- and edge-potential-related *Greens* functions are shown in one plot, due to the later discussed choice of f. Furthermore, the results for

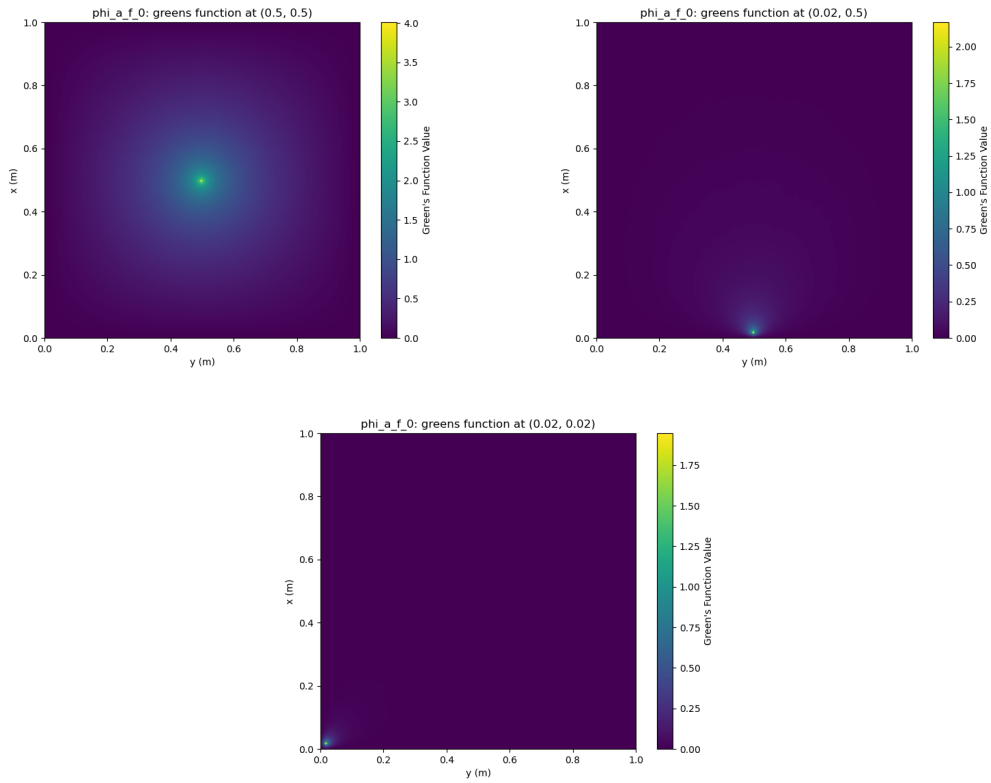

Figure 1: Green's function for points $(0.5, 0.5)$, $(0.02, 0.5)$ and $(0.02, 0.02)$.

all possible combinations in task 4, including the $f = 0$ case, are printed out, toghether with the run-time, the standard deviation and the *Gauss-Seidel* result. Such are listed in Tables 1, 2 and 3 for points $(0.5, 0.5)$, $(0.02, 0.5)$ and $(0.02, 0.02)$. The grid size was set to $N = 200$ and the number of walkers per core to 100000 (16000000 total walks). The grid size was chosen, so that the *Gauss-Seidel* solution does not significantly change when increasing N further. The number of walkers per core was chosen, so that the standard deviation is small enough and both solutions are close enough.

Table 1: Simulation results for point (0.5, 0.5)

ϕ_0 / f	Runtime	Std. Dev.	Random Walk	Gauss-Seidel	Difference
(a) / 0	98.0060	0.2078	100.0000	100.0000	0.0000
(b) / 0	97.6400	0.2079	-0.1255	0.0000	0.1255
(c) / 0	98.2562	0.2079	0.5216	0.8388	0.3172
(a) / (a)	95.5251	0.2078	102.9446	102.9467	0.0021
(b) / (a)	100.9842	0.2079	2.9202	2.9467	0.0265
(c) / (a)	98.2108	0.2079	3.2973	3.7855	0.4882
(a) / (b)	95.9537	0.2080	100.1478	100.1477	0.0001
(b) / (b)	100.8678	0.2080	0.1383	0.1477	0.0094
(c) / (b)	98.9432	0.2080	1.1456	0.9865	0.1591
(a) / (c)	95.9975	0.2080	100.0507	100.0507	0.0000
(b) / (c)	101.7416	0.2079	0.0371	0.0507	0.0136
(c) / (c)	98.3983	0.2079	0.7549	0.8895	0.1346

Table 2: Simulation results for point (0.02, 0.02)

Experiment	Runtime	Std. Dev.	Random Walk	Gauss-Seidel	Difference
(a) / 0	56.5228	0.0160	100.0000	100.0000	0.0000
(b) / 0	55.3698	0.0159	0.0132	-0.0000	0.0132
(c) / 0	55.2412	0.0160	99.8714	99.9005	0.0291
(a) / (a)	56.0235	0.0159	100.0229	100.0228	0.0001
(b) / (a)	55.2039	0.0160	0.0255	0.0228	0.0027
(c) / (a)	55.2322	0.0160	99.9101	99.9233	0.0132
(a) / (b)	55.8051	0.0160	100.0020	100.0019	0.0000
(b) / (b)	55.2051	0.0161	0.0246	0.0019	0.0227
(c) / (b)	55.1880	0.0161	99.8540	99.9025	0.0485
(a) / (c)	55.7681	0.0161	100.0001	100.0001	0.0000
(b) / (c)	55.3402	0.0161	-0.0084	0.0001	0.0085
(c) / (c)	55.2358	0.0159	99.9167	99.9006	0.0161

Table 3: Simulation results for point (0.02, 0.5)

Experiment	Runtime	Std. Dev.	Random Walk	Gauss-Seidel	Difference
(a) / 0	58.3636	0.0537	100.0000	100.0000	0.0000
(b) / 0	58.4673	0.0539	94.9439	94.9692	0.0254
(c) / 0	58.6143	0.0539	-1.4389	-1.4024	0.0365
(a) / (a)	58.3290	0.0540	100.2000	100.1991	0.0009
(b) / (a)	58.7982	0.0538	95.1776	95.1683	0.0093
(c) / (a)	58.6602	0.0538	-1.2524	-1.2033	0.0492
(a) / (b)	57.9417	0.0538	100.0145	100.0145	0.0000
(b) / (b)	58.2354	0.0538	94.9961	94.9837	0.0124
(c) / (b)	58.5365	0.0538	-1.3421	-1.3879	0.0458
(a) / (c)	58.0020	0.0537	100.0015	100.0015	0.0000
(b) / (c)	58.4828	0.0538	94.9829	94.9708	0.0121
(c) / (c)	58.5473	0.0536	-1.3949	-1.4008	0.0060

A accuracy of 4 decimals was not achievable, due to the long computational time. The errors (differences) lie mostly in the standard deviation of the greens function. The wall clock time for the entire program, yielding all above results, was 3247.5 seconds, which is about 54 minutes.

Three main alterations to the naive implementations were made.

1. Instead of working with f, the algorithm uses $h^2 f$. This causes the *Greens* function to be just the probability of the random walk hitting a certain point, inside the grid and at the boundary.

2. The package *njit* from *numba* is used to compile the random walk function, avoiding the overhead of the python interpreter. This results in a significant speed up.

3. The package *parallel-statistics* was used to calculate mean, i.e. the *Greens* function, and variance of the random walk. Note, that due to heavy memory occupation, the mean and variance were calculated in batches.

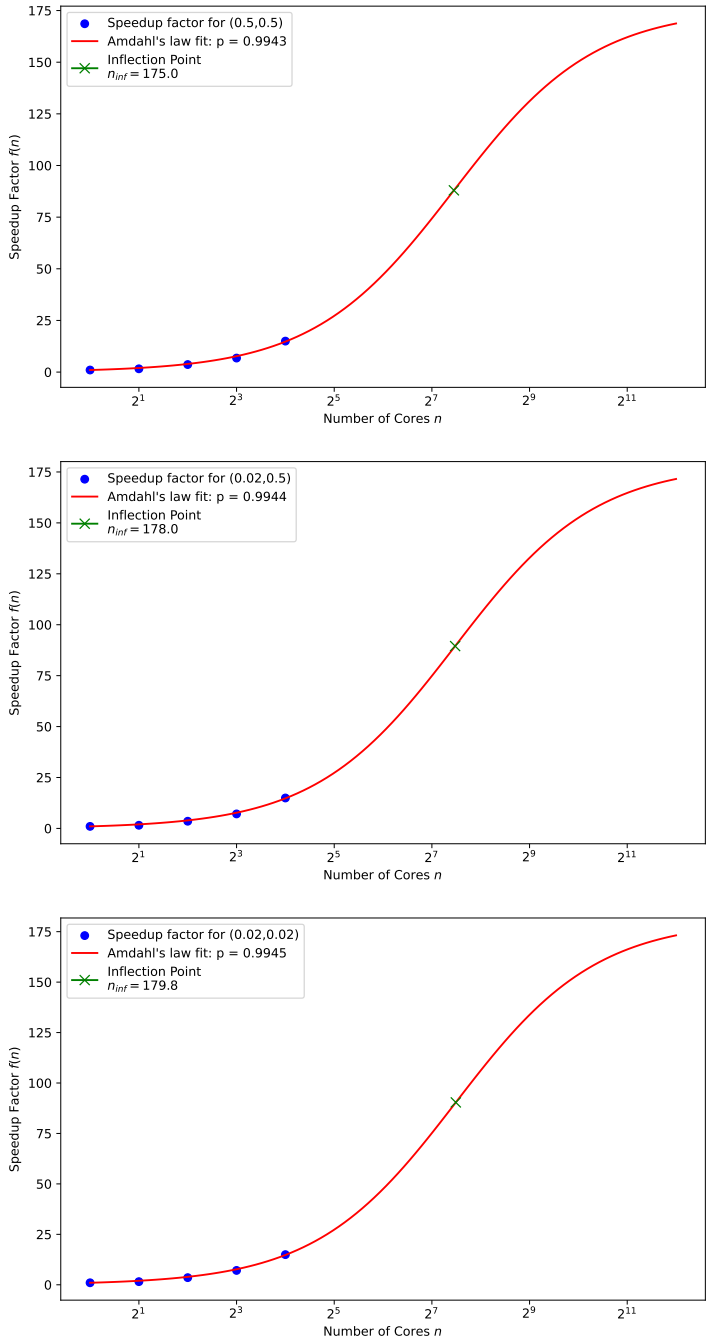

Figure 2: Amdahl's law for points $(0.5, 0.5)$, $(0.02, 0.5)$ and $(0.02, 0.02)$. The times for using only one core are given by 157.0015, 86.3661 and 90.2549 seconds, respectively.

In Figure 2 the speed up factors, for $N = 200$, 160000 total walks and varying starting points $(0.5, 0.5)$, $(0.02, 0.5)$, $(0.02, 0.02)$, are shown. This only includes the computation of the greens function, not the solution of the poisson equation. Three runtimes were recorded and averaged for each starting point.

Note, that the duration of the walk depends significantly on the starting point, due to the varying distance to the boundary. However, the parallelization p is the same, as shown below. *Amdahl's* law

$$f(n) = \frac{1}{1 - p + \frac{p}{n}} \tag{1}$$

is fitted to the data, yielding the respective values for p. The inflection point could not be observed, since only 16 cores were available. The theoretical value for the inflection point is given by

$$n_{inf} = \frac{p}{1-p} = 177.6 \approx 2^{7.47} \tag{2}$$

cores, using the average value $\bar{p} = 0.9944$.

When using *pylint*, the flag that the programm is named *assignment-3* was ignored, aswell as some snake-case and redifining flags, when the context was obvious.